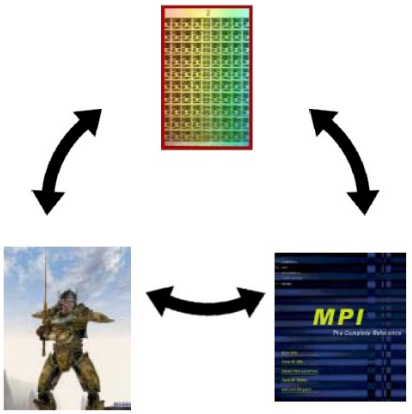
29

The Likely HPC Ecosystem from 202x+

CPU + GPU = future many-core driven by commercial applications



MPI+(autotuning, PGAS, ??)

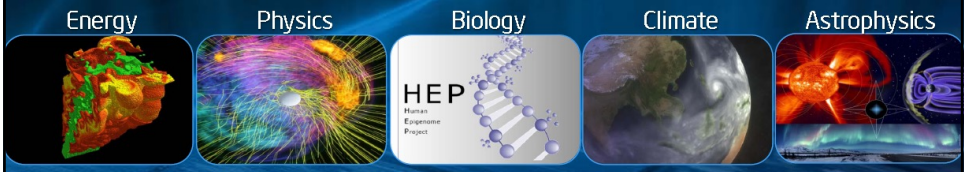
Next generation "clusters" with many-core or hybrid nodes

29

Exascale: *The Next Frontier*

Challenges

- Power – energy / operation of computation, data transport, memory
- Threading software to millions/billions of threads
- Memory/Storage capacity and bandwidth
- Managing high-node count systems in the existence of failures (MTBF)
- Affordability



Intel committed to solving the challenges of Exascale

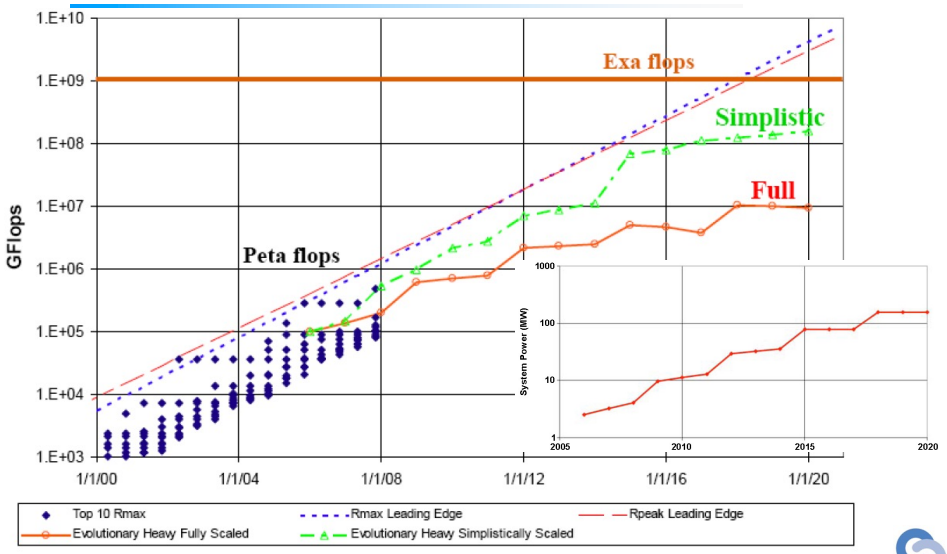
30

DARPA Exascale Study

- Commissioned by DARPA to explore the challenges for Exaflop computing
- Two models for future performance growth
  - **Simplistic**:
    - ITRS (International Technology Roadmap for Semiconductors) roadmap
    - Power for memory grows linear with # of chips
    - Power for interconnect stays constant
  - **Fully scaled**: same as simplistic, but memory and router power grow with peak flops per chip



Won't reach Exascale this way



From Peter Kogge, DARPA ExascaleStudy



33

Extrapolating to Exaflop/s

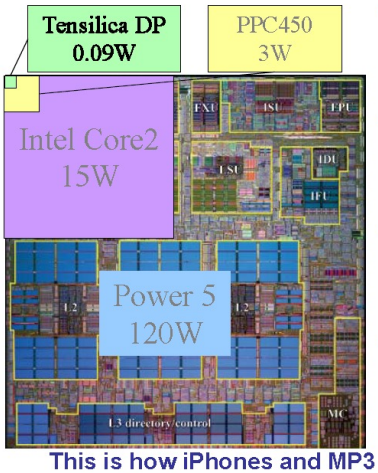
	BlueGene/L (2005)	Exaflop Directly scaled	Exaflop compromise using expected technology	Assumption for "compromise guess"
Node Peak Perf	5.6GF	20TF	20TF	Same node count (64k)
hardware concurrency/node	2	8000	1600	Assume 3.5GHz
System Power in Compute Chip	1 MW	3.5 GW	35 MW	100x improvement (very optimistic)
Link Bandwidth (Each unidirectional 3-D link)	1.4Gbps	5 Tbps	1 Tbps	Not possible to maintain bandwidth ratio.
Wires per unidirectional 3-D link	2	400 wires	80 wires	Large wire count will eliminate high density and drive links onto cables where they are 100x more expensive. Assume 20 Gbps signaling
Pins in network on node	24 pins	5,000 pins	1,000 pins	20 Gbps differential assumed. 20 Gbps over copper will be limited to 12 inches. Will need optics for in rack interconnects. 10Gbps now possible in both copper and optics.
Power in network	100 KW	20 MW	4 MW	10 mW/Gbps assumed. Now: 25 mW/Gbps for long distance (greater than 2 feet on copper) for both ends one direction. 45mW/Gbps optics both ends one direction. + 15mW/Gbps of electrical Electrical power in future: separately optimized links for power.
Memory Bandwidth/node	5.6GB/s	20TB/s	1 TB/s	Not possible to maintain external bandwidth/Flop
L2 cache/node	4 MB	16 GB	500 MB	About 6-7 technology generations
Data pins associated with memory/node	128 data pins	40,000 pins	2000 pins	3.2 Gbps per pin
Power in memory I/O (not DRAM)	12.8 KW	80 MW	4 MW	10 mW/Gbps assumed. Most current power in address bus. Future probably about 15mW/Gbps maybe get to 10mW/Gbps (2.5mW/Gbps is c*v^2*f for random data on data pins) Address power is higher.
QCD CG single iteration time	2.3 msec	11 usec	15 usec	Requires: 1) fast global sum (2 per iteration) 2) hardware offload for messaging (Driverless messaging)

Source: David Turek, IBM

33

34

Design for Low Power: More Concurrency



- Cubic power improvement with lower clock rate due to V^2F
- Slower clock rates enable use of simpler cores
- Simpler cores use less area (lower leakage) and reduce cost
- Tailor design to application to reduce waste

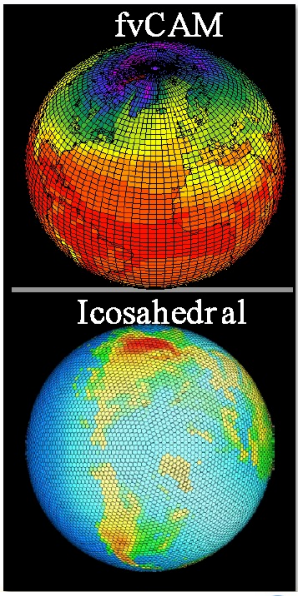
This is how iPhones and MP3 players are designed to maximize battery life and minimize cost

34

Green Flash: Ultra-Efficient Climate Modeling

35

- An alternative route to exascale computing
  - Exascale science questions are identified
  - The idea is to target specific machine designs to each of these questions
    - Possible because of new technologies driven by the consumer market
- We want to turn the process around
  - Ask "What machine do we need to answer a question?"
  - Not "What can we answer with that machine?"
- Goal: influence the HPC industry by evaluating a prototype design





35

36

Green Flash Strawman System Design

We examined three different approaches

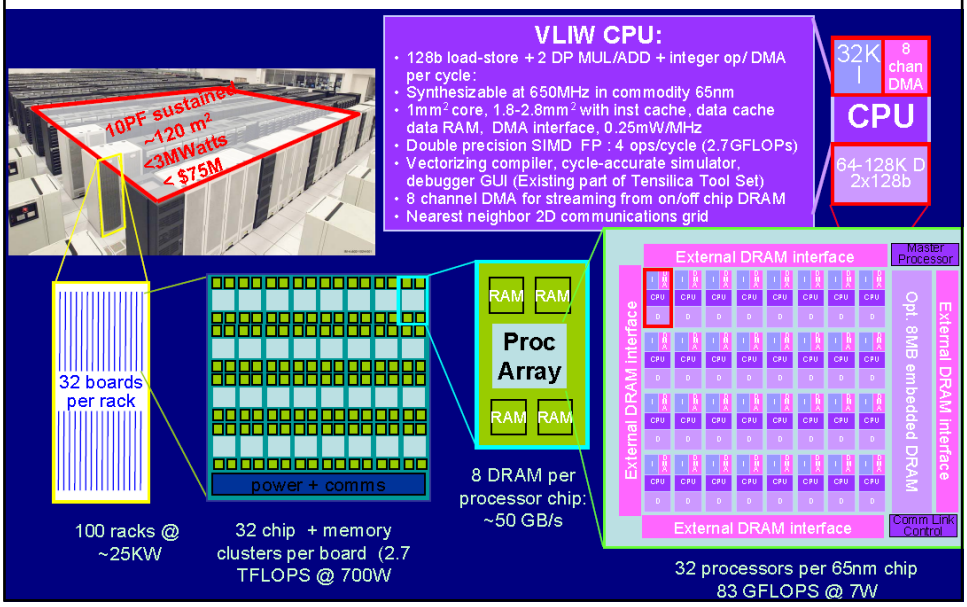
Computation: 10 PFlops sustained, ~200 PFlops peak

- **AMD Opteron**: Commodity approach, lower efficiency for scientific applications offset by cost efficiencies of mass market
- **BlueGene**: Generic embedded processor core and customize system-on-chip (SoC) to improve power efficiency for scientific applications
- **Tensilica XTensa**: Customized embedded CPU w/SoC provides further power efficiency benefits but maintains programmability

Processor	Clock	Peak/ Core (Gflops)	Cores/ Socket	Sockets	Cores	Power	Cost 2008
AMD Opteron	2.8GHz	5.6	2	890K	1.7M	179 MW	$1B+
IBM BG/P	850MHz	3.4	4	740K	3.0M	20 MW	$1B+
Green Flash / Tensilica XTensa	650MHz	2.7	32	120K	4.0M	3 MW	$75M

36

Climate System Design Concept



37

Summary on Green Flash

- Exascale computing is vital for numerous key scientific areas
- We propose a new approach to high-end computing that enables transformational changes for science
- Research effort: study feasibility and share insight w/ community
- This effort will augment high-end general purpose HPC systems
  - Choose the science target first (climate in this case)
  - Design systems for applications (rather than the reverse)
  - Leverage power efficient embedded technology
  - Design hardware, software, scientific algorithms together using hardware emulation and auto-tuning
  - Achieve exascale computing sooner and more efficiently

Applicable to broad range of exascale-class applications



38

39

What about 1 million cores?

- What are applications developers concerned about?
- … but before we answer this question, the more interesting question is …
  - 1000 cores on the laptop ?
- What are commercial applications developers going to do with it?



39

40

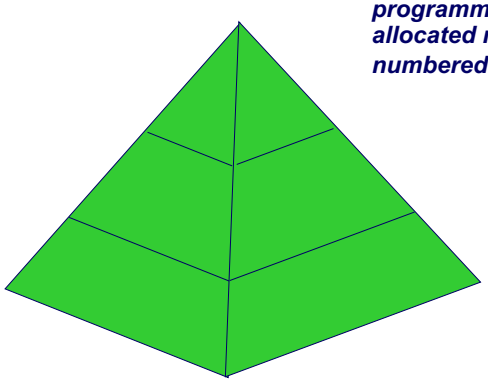
Summary

- Major Challenges are ahead for HPC
  - Power
  - Parallelism
  - …and many others not discussed here
- We will need completely new approaches and technologies to reach the Exascale level
- This opens up a unique opportunity for science applications to lead extreme scale systems development



40

The Pyramid of Parallel Programming



Thousand-node systems with MPI-style programming, >1-10 PFLOPS, $M, allocated machine time (programmers numbered in hundreds)

Hundred-core systems with CUDA-style programming, 1-5 TFLOPS, $K, machines widely availability (programmers numbered in 10s of thousands)

Hundred-core systems with MatLab-style programming, 10-100 GFLOPS, $K, machines widely available (programmers numbered in millions)



Foster's Design Methodology

- From *Designing and Building Parallel Programs* by Ian Foster
- Four Steps:
  - **Partitioning**
    - Dividing computation and data
  - **Communication**
    - Sharing data between computations
  - **Agglomeration**
    - Grouping tasks to improve performance
  - **Mapping**
    - Assigning tasks to processors/threads



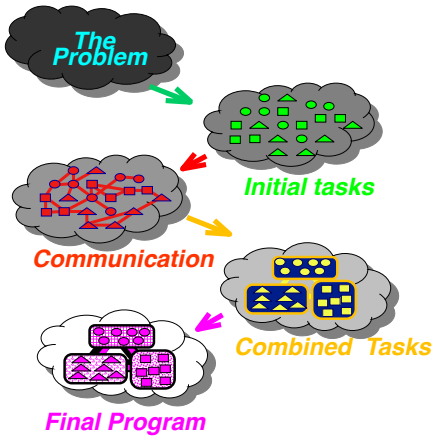
Parallel Algorithm Design: PCAM

- **Partition**: Decompose problem into fine-grained tasks to maximize potential parallelism
- **Communication**: Determine communication pattern among tasks
- **Agglomeration**: Combine into coarser-grained tasks, if necessary, to reduce communication requirements or other costs
- **Mapping**: Assign tasks to processors, subject to tradeoff between communication cost and concurrency



Designing Threaded Programs

- **Partition**
  - Divide problem into tasks
- **Communicate**
  - Determine amount and pattern of communication
- **Agglomerate**
  - Combine tasks
- **Map**
  - Assign agglomerated tasks to created threads





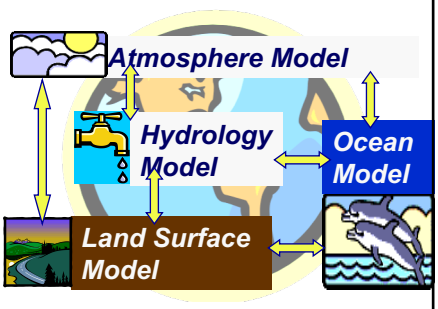
Parallel Programming Models

- Functional Decomposition
  - Task parallelism
  - Divide the computation, then associate the data
  - Independent tasks of the same problem
- Data Decomposition
  - Same operation performed on different data
  - Divide data into pieces, then associate computation

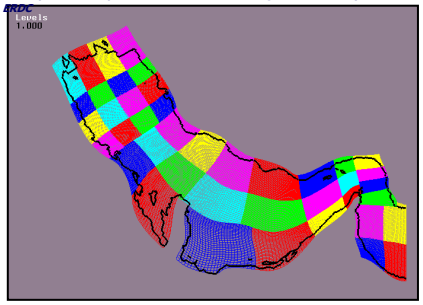


Decomposition Methods

- Functional Decomposition
  - Focusing on computations can reveal structure in a problem



Grid reprinted with permission of Dr. Phu V. Luong, Coastal and Hydraulics Laboratory, ERDC



Domain Decomposition

- *Focus on largest or most frequently accessed data structure*
- *Data Parallelism*
  - *Same operation applied to all data*



Example: Computing Pi



- We want to compute π
- One method: method of darts*
- Ratio of area of square to area of inscribed circle proportional to π



Disclaimer: this is a TERRIBLE way to compute π. Don't even think about doing it this way in real life!



Method of Darts

- Imagine dartboard with circle of radius R inscribed in square
- Area of circle $= \pi R^2$
- Area of square $= (2R)^2 = 4R^2$
- Area of circle / Area of square $= \frac{\pi R^2}{4R^2} = \frac{\pi}{4}$





Method of Darts



- So, ratio of areas proportional to π
- How to find areas?
  - Suppose we threw darts (**completely randomly**) at dartboard
  - Could count number of darts landing in circle and total number of darts landing in square
  - Ratio of these numbers gives approximation to ratio of areas
  - Quality of approximation increases with number of darts
    - $\pi = 4 \times \frac{\text{\# darts inside circle}}{\text{\# darts thrown}}$



Method of Darts

- Okay, but how in the world do we simulate this experiment on a computer?
  - Decide on length R
  - Generate pairs of random numbers (x, y) so that $-R \leq x, y \leq R$
  - If (x, y) within circle (i.e. if $(x^2+y^2) \leq R^2$), add one to tally for inside circle
  - Lastly, find ratio



51

Parallelization Strategies

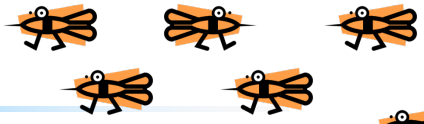
- What tasks independent of each other?
- What tasks must be performed sequentially?
- Using PCAM parallel algorithm design strategy



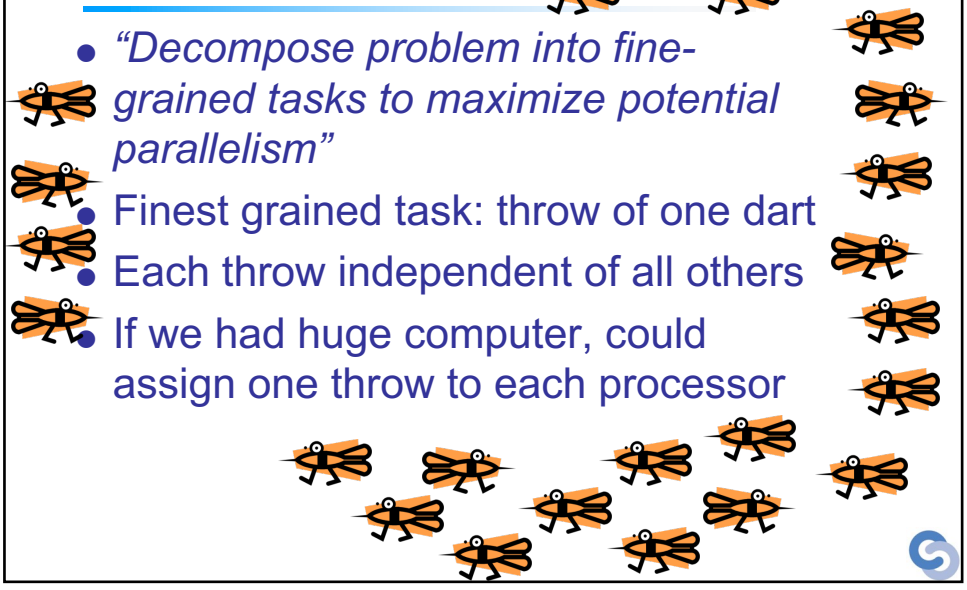
51

52

Partition



- *"Decompose problem into fine-grained tasks to maximize potential parallelism"*
- Finest grained task: throw of one dart
- Each throw independent of all others
- If we had huge computer, could assign one throw to each processor



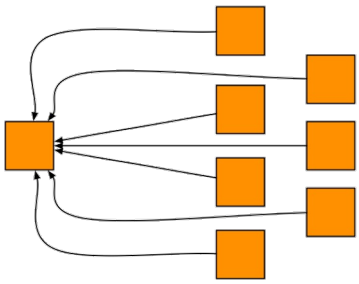


52

Communication

"Determine communication pattern among tasks"

- Each processor throws dart(s) then sends results back to manager process







Agglomeration

"Combine into coarser-grained tasks, if necessary, to reduce communication requirements or other costs"

- To get good value of π, must use millions of darts
- We don't have millions of processors available
- Furthermore, communication between manager and millions of worker processors would be very expensive
- Solution: divide up number of dart throws evenly between processors, so each processor does a share of work



55

Mapping

"Assign tasks to processors, subject to tradeoff between communication cost and concurrency"

- Assign role of "manager" to processor 0
- Processor 0 will receive tallies from all the other processors, and will compute final value of π
- Every processor, including manager, will perform equal share of dart throws

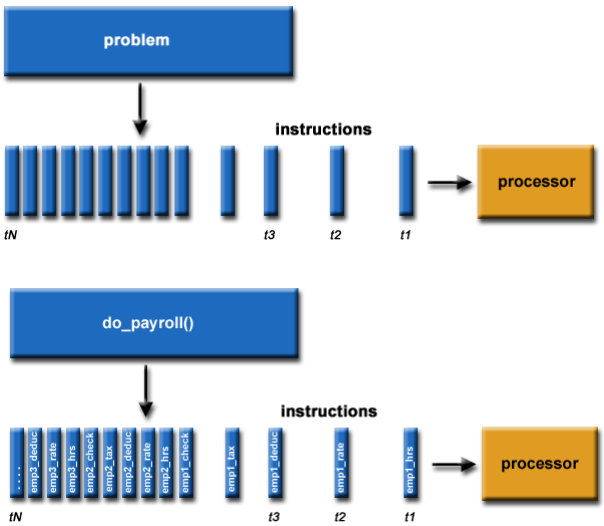




55

56

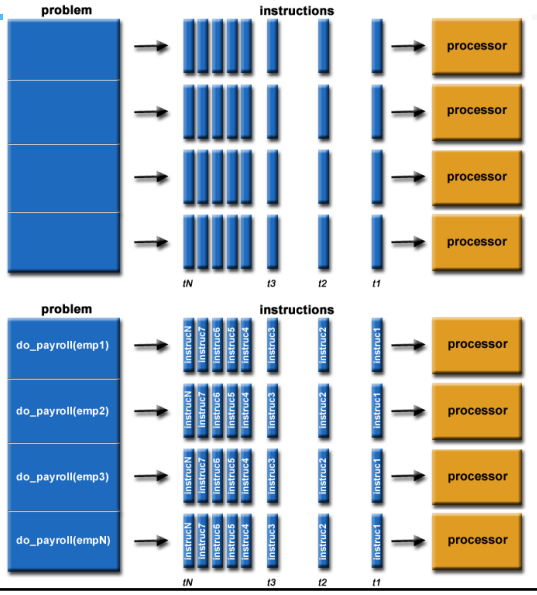
Serial Approach



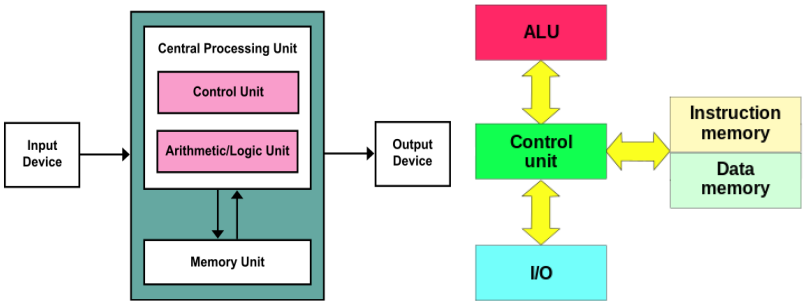


56

Parallel Approach



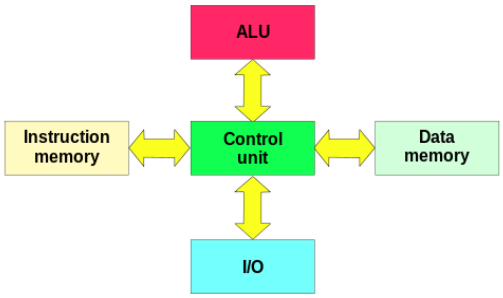
Von Neumann Architectures



- Instruction Flow
- Data Flow
- Single Address Space for Intsr & Data
- Low I/O Capabilities
- General computing use

59

Harvard Architecture



- **Separate** Instruction & Data Flow
- Separate Memory Spaces for Instructions & Data
- Used for DSPs



59

60

Most Current Computing Systems

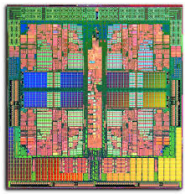
- Are parallel at multiple HW levels
  - L1 / L2
  - Prefetching
  - Pipeline
  - Floating point operations
  - GPU processing
  - Multi-cores
  - Many-cores
  - Multi-threading (HW)

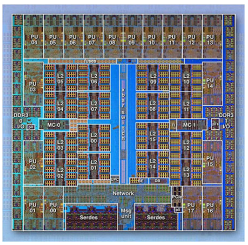


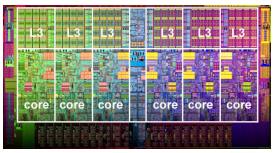
60

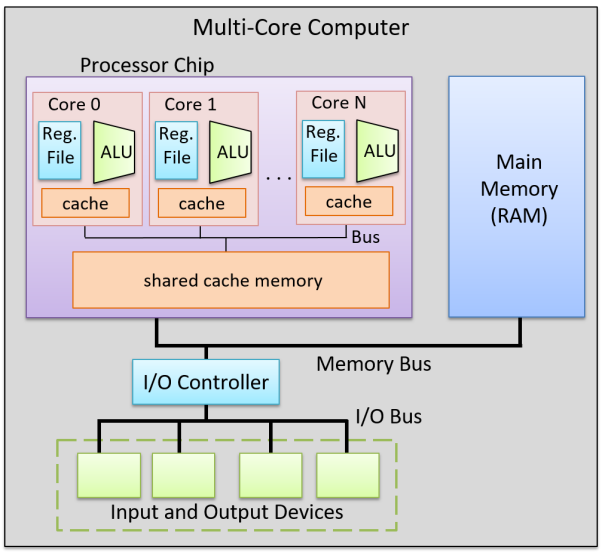
61

Multi-cores Processors





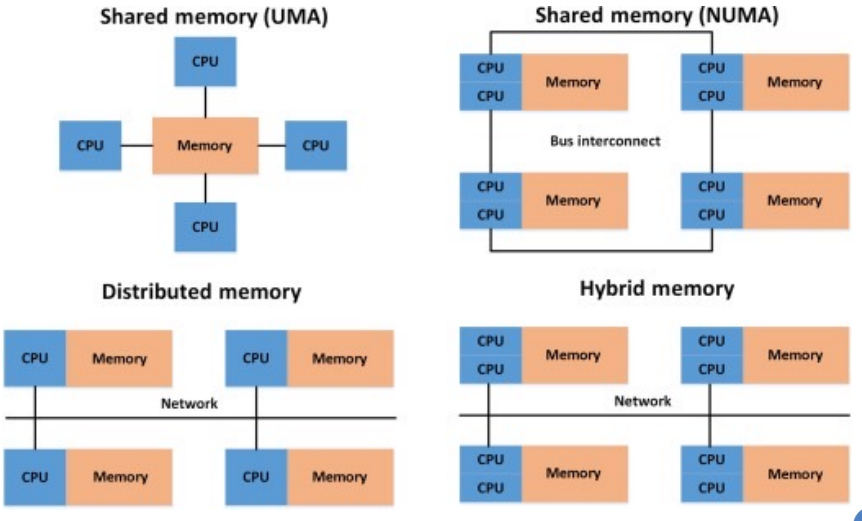




61

62

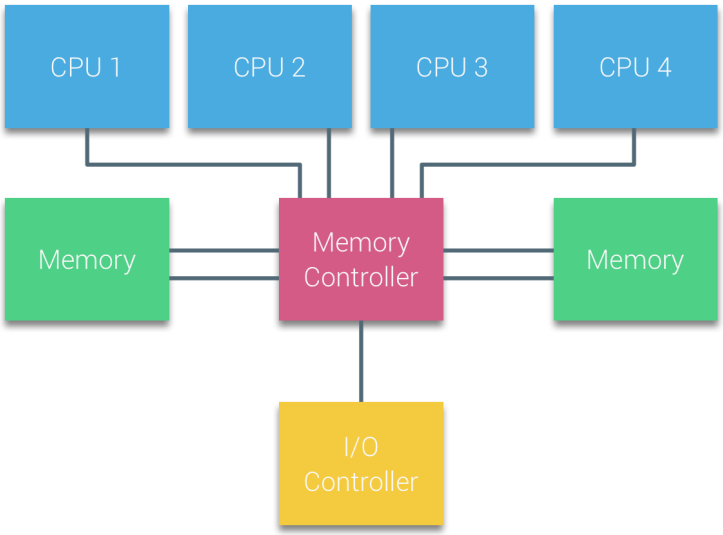
Evolution & Variants of Systems



62

63

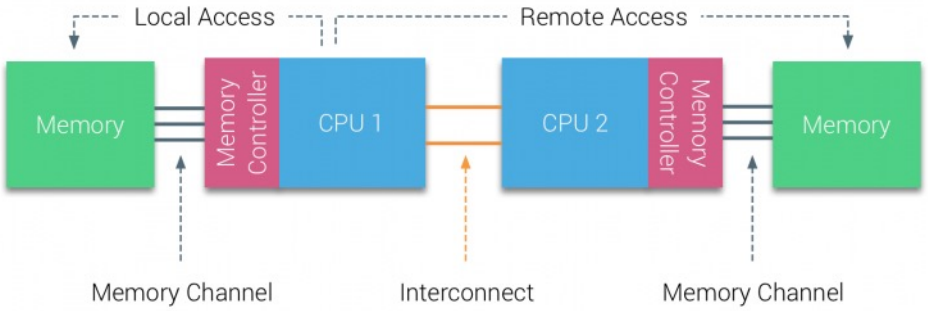
UMA – Uniform Memory Access



63

64

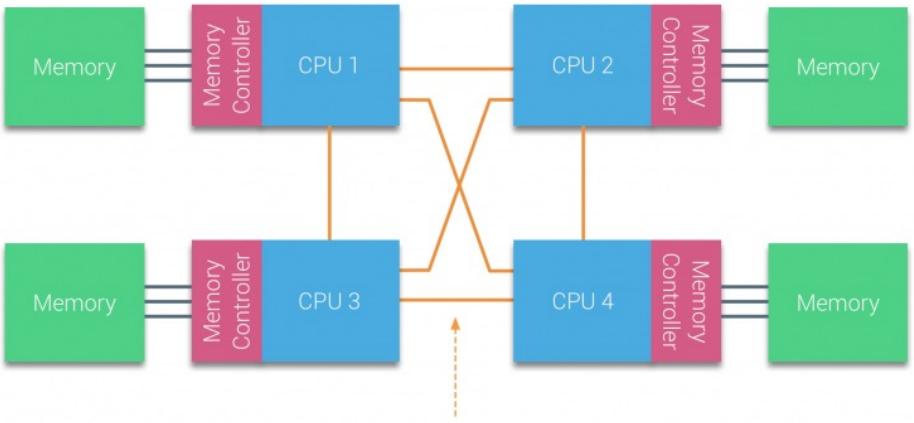
NUMA – Non-Uniform MA



64

65

Inter-processor Communication



HT
NVlink



65

66

UMA vs. NUMA

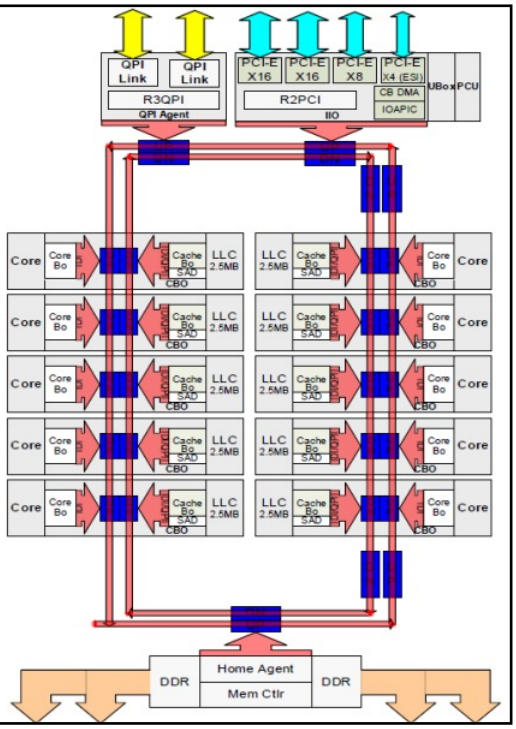
Uniform memory access	Non-uniform memory access
Offers limited bandwidth.	Offers relatively more bandwidth than UMA.
Since it uses a single memory controller, it is slower than NUMA.	It uses multiple memory controls to speed up operations compared to UMA.
Memory access time is equal or balanced.	Memory access time is unequal.
It is mainly used in time-sharing and general-purpose applications.	It offers better performance and speed, making it suitable for real-time and time-critical applications.



66

CC-NUMA

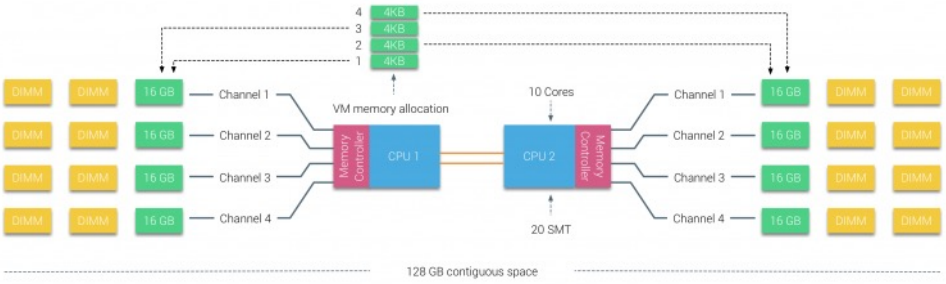
- Coherennce at the Cache Level
- Coherence solved in HW by snooping algorithm



67

68

SUMA – Sufficiently UMA



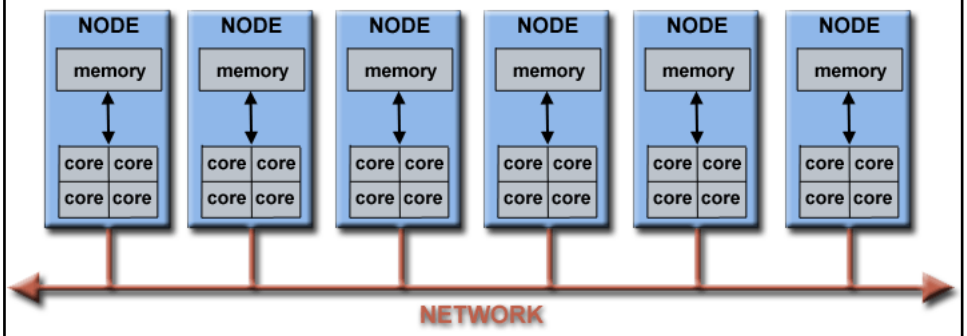
- Splits a NUMA in 4k segments
- Maps segments in round-robin across nodes
- Access time is uniform & depends on QPI/HT contention

NUMA	Memory Node 0	Memory Node 1	–	SUMA	Memory Node 0	Memory Node 1
Socket 0	75.7	132.0	–	Socket 0	105.5	106.4
Socket 1	131.9	75.8	–	Socket 1	106.0	104.6

68

69

Clusters of Computers





69

70

Cluster@CS – 2005-2012



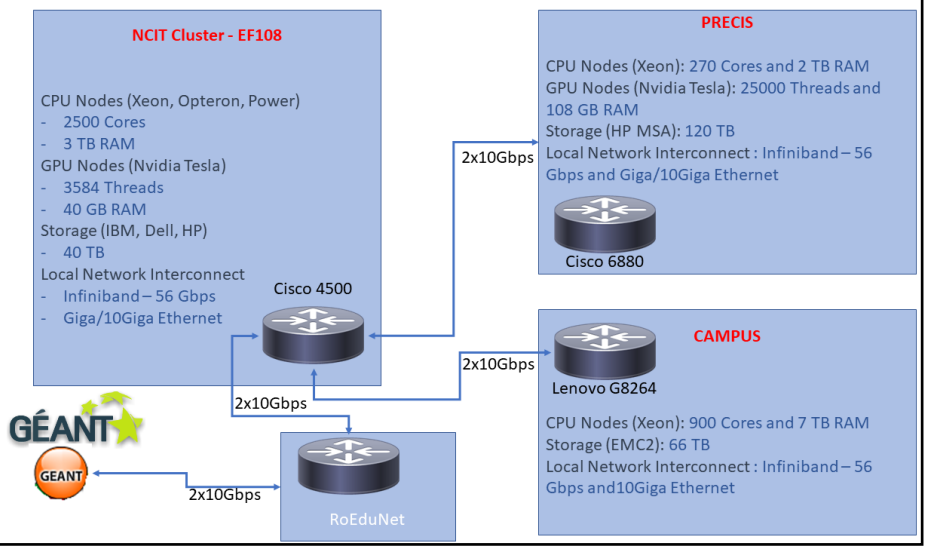




70

71

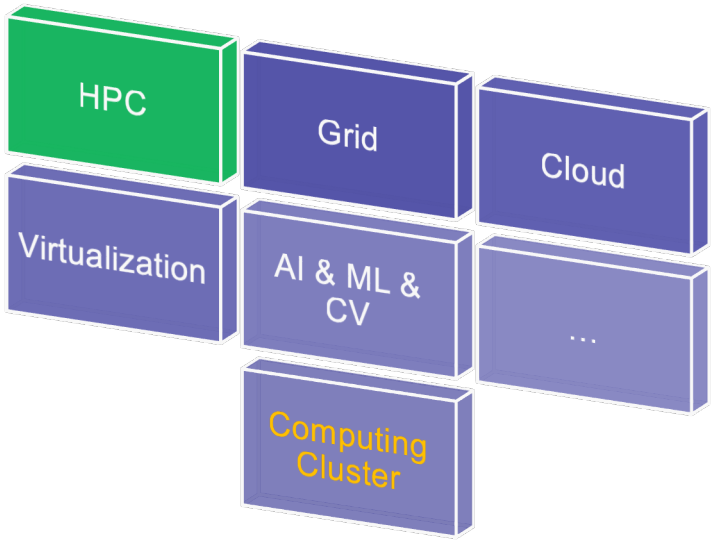
Clusters@CS – 2015-202x



71

72

Cluster Services



72

Our training & user domains

Hardware needs people to function

MPI OpenMP PThreads

CUDA / Multicore Computing

Summer Schools

Profiling / Tuning

Diploma

Master R&D

first sem. *second sem.* Σ *HPC / PP / Cluster / Cloud / Q*

3rd year of study *4th year of study* *2 years*



Projects Research PhDs

3-5 years

